

Youtube Clone Backend – Deep A-to-Z Documentation

1. High-level overview

This repository implements a **Node.js + Express + MongoDB** backend for a “YouTube-like” platform.

Major backend responsibilities

1. **Auth** (JWT access token + refresh token; tokens stored in cookies)
2. **User profiles** (register/login/logout, update account details)
3. **Media & uploads** (avatar/cover uploads to Cloudinary; temp upload via Multer)
4. **Video interactions** (a controller skeleton exists for video CRUD, but many methods are TODO)
5. **Comments, likes, subscriptions, playlists** (controllers are largely TODO/skeleton)
6. **Watch history** and **channel profile aggregation** using MongoDB aggregation pipelines

Request lifecycle (common)

Client → Express route → Middleware (auth/multer) → Controller → Mongoose model/aggregation → JSON response

2. Tech stack

Runtime & framework

- **Node.js** (ESM modules: `"type": "module"`)
- **Express** (`express` v5.2.1)

Database

- **MongoDB**
- **Mongoose** v9
- MongoDB aggregations are used (notably in `user.controller.js`)

Auth

- **JWT**(`jsonwebtoken`)
- **Cookies**(`cookie-parser`)

File upload & third-party

- **Multer** for local temp storage
- **Cloudinary** for media storage

Project conventions used in this backend

- Async route controllers wrapped with `asyncHandler` (promise wrapper)
 - Consistent response envelope class:
 - `ApiResponse` (sets `success` based on status code)
 - Error class:
 - `ApiError` (includes `statusCode` , `message` , `errors`)
-

3. Folder structure (Youtube_Clone_Backend)

Youtube_Clone_Backend/

src/

app.js

index.js

constants.js

controllers/

user.controller.js

video.controller.js

comment.controller.js

like.controller.js

playlist.controller.js

subscription.controller.js

dashboard.controller.js

tweet.controller.js

healthcheck.controller.js

routes/

user.routes.js

```
healthcheck.routes.js
video.routes.js
comment.routes.js
like.routes.js
playlist.routes.js
subscription.routes.js
dashboard.routes.js
tweet.routes.js
```

```
middlewares/  
  auth.middleware.js  
  multer.middleware.js
```

```
models/  
  user.model.js  
  video.model.js  
  comment.model.js  
  like.model.js  
  playlist.model.js  
  subscription.model.js  
  tweet.model.js
```

```
db/  
  index.js
```

```
utils/  
  ApiError.js  
  ApiResponse.js  
  AsyncHandler.js  
  cloudinary.js
```

(also: package.json, README.md, .env expected)

4. Entry points

4.1 `src/index.js`

Responsibilities:

- Loads environment variables via `dotenv.config()`

- Connects to MongoDB (calls `connectdb()`)
- Starts Express `app.listen()` after DB connection

Key behavior:

- Logs loaded env `process.env.MONGODB_URI`
- Server port uses `process.env.PORT || 8000`

4.2 `src/app.js`

Responsibilities:

- Creates Express app
- Installs middleware
- Mounts routers under `/api/v1/...`

Middleware installed:

- `cors({ origin: process.env.CORS_ORIGIN || "*", credentials: true })`
- `express.json({ limit: "16kb" })`
- `express.urlencoded({ extended: true, limit: "16kb" })`
- `express.static("public")`
- `cookieParser()`

Routes mounted:

- `/api/v1/healthcheck`
- `/api/v1/users`
- `/api/v1/tweets`
- `/api/v1/subscriptions`
- `/api/v1/videos`
- `/api/v1/comments`
- `/api/v1/likes`
- `/api/v1/playlist`
- `/api/v1/dashboard`

5. Database layer

5.1 `src/db/index.js`

Responsibilities:

- Connect to MongoDB using mongoose

Implementation detail:

- Uses `mongoose.connect( ${process.env.MONGODB_URI}/${DB_NAME} )`
- Imports `DB_NAME` from `../constants.js`
- On success logs host from the connection info
- On failure prints error and exits(`process.exit(1)`)

5.2 `src/constants.js`

- `export const DB_NAME = "Project0"`
-

6. Middleware

6.1 `middlewares/auth.middleware.js`

Exports:

- `verifyJWT`

Behavior:

1. Extracts JWT from either:
 - `req.cookies?.accessToken` , OR
 - `req.header("Authorization")` with Bearer <token>
2. If no token → throws `ApiError(401, "Unauthorized request")`
3. Verifies token via:
 - `jwt.verify(token, process.env.ACCESS_TOKEN_SECRET)`
4. Loads user:
 - `User.findById(decoded._id).select("-password -refreshToken")`
5. If no user → throws `ApiError`
6. Sets `req.user = user` and calls `next()`

Notes / caveats:

- Error handling wraps everything into `throw new ApiError(401, ...)`
- Token payload expects `_id` key (as generated in `user.model.js` methods)

6.2 middlewares/multer.middleware.js

Exports:

- `upload`

Behavior:

- Uses Multer disk storage:
 - destination: `./public/temp`
 - filename: original filename

Used for:

- Avatar and coverimage uploads (register and update endpoints)
 - Single-file uploads via `upload.single(...)` and multipart via `upload.fields(...)`
-

7. Utils

7.1 utils/ApiResponse.js

Class:

- `new ApiResponse(statusCode, data, message = "Success")`

Sets:

- `this.success = statusCode < 400`

7.2 utils/ApiError.js

Class:

- Extends `Error` :
 - `statusCode, data, message, success=false, errors`
- If `stack` argument provided, uses it; else uses `Error.captureStackTrace`

7.3 utils/AsyncHandler.js

Function:

- `asyncHandler(requesthandler)` returns an Express middleware that:
 - `Promise.resolve(requesthandler(req, res, next)).catch(next)`

7.4 utils/cloudinary.js

Exports:

- `uploadoncloudinary(localfilepath)`

Behavior:

- Configures Cloudinary using environment variables:
 - `CLOUDINARY_CLOUD_NAME`
 - `CLOUDINARY_API_KEY`
 - `CLOUDINARY_API_SECRET`
- Uploads local file:
 - `cloudinary.uploader.upload(localfilepath, { resource_type: "auto" })`
- After successful upload:
 - Deletes local temp file with `fs.unlink(localfilepath)`
- On failure:
 - Deletes local temp file if present
 - Throws error

Note: In controllers there is a naming mismatch (`uploadOnCloudinary` vs `uploadoncloudinary`). The user controller uses `uploadoncloudinary` correctly.

8. Models (Mongoose)

8.1 models/user.model.js

Schema fields:

- `username` :
 - String, required, unique, lowercase, trim, indexed
- `email` :
 - String, required, unique, lowercase, trim
- `fullname` :
 - String, required, lowercase, trim

- `avatar` : String, required
- `coverimage` : String (optional)
- `watchhistory` : array of Video ObjectId refs
- `password` :
 - String, required
- `refreshToken` : String

Hooks & methods:

- `pre("save")` :
 - If password modified → bcrypt hash with salt rounds 10
- `methods.isPasswordcorrect(password)` :
 - `bcrypt.compare`
- `methods.generateAccesstoken()` :
 - JWT includes `_id`, `email`, `username`, `fullname` (fullName uses `this.username` in payload)
 - signed with `ACCESS_TOKEN_SECRET`
 - expiry uses `process.env.ACCESS_TOKEN_EXPIRY`
- `methods.generateRefreshToken()` :
 - JWT includes `_id`
 - signed with `REFRESH_TOKEN_SECRET`
 - expiry uses `process.env.REFRESH_TOKEN_EXPIRY`

Exports:

- `export const User = mongoose.model("User", userSchema)`

8.2 models/video.model.js

Schema:

- `videofile` (String, required)
- `thumbnail` (String, required)
- `title` (String, required)
- `description` (String, required)
- `duration` (Number, required)
- `views` (Number default 0)
- `ispublish` (Boolean default true)
- `owner` (ObjectId ref "User")

Plugin:

- `mongoose-aggregate-paginate-v2`

Exports:

- `export const Video = mongoose.model("Video", videoSchema)`

8.3 `models/comment.model.js`

Schema:

- `content` : String required
- `video` : ObjectId ref "Video"
- `owner` : ObjectId ref "User"

Plugin:

- `mongoose-aggregate-paginate-v2`

Exports:

- `export const Comment = mongoose.model("Comment", commentSchema)`

8.4 `models/like.model.js`

Schema fields:

- `video` : ObjectId ref "Video" (optional)
- `comment` : ObjectId ref "Comment" (optional)
- `tweet` : ObjectId ref "Tweet" (optional)
- `likedBy` : ObjectId ref "User"

Exports:

- `export const Like = mongoose.model("Like", likeSchema)`

8.5 `models/subscription.model.js`

Schema:

- `subscriber` : ObjectId ref "User"
- `channel` : ObjectId ref "User"

Exports:

- `export const Subscription = mongoose.model("Subscription", subscriptionSchema)`

8.6 `models/playlist.model.js`

Schema:

- `name` : String required
- `description` : String required
- `videos` : array of Video refs
- `owner` : ObjectId ref "User"

Exports:

- `export const Playlist = mongoose.model("Playlist", playlistSchema)`

8.7 `models/tweet.model.js`

Schema:

- `content` : String required
- `owner` : ObjectId ref "User"

Exports:

- `export const Tweet = mongoose.model("Tweet", tweetSchema)`
-

9. Controllers and routes

Important: Many controllers are **TODO skeletons**. The user controller contains real logic for auth, profile/channel aggregation, and watch history.

9.1 Healthcheck

Controller: `controllers/healthcheck.controller.js`

- `healthcheck` is TODO.

Route: `routes/healthcheck.routes.js`

- GET /api/v1/healthcheck/
- Calls `healthcheck`

9.2 User controller (fully implemented portions)

File: `controllers/user.controller.js`

Exports:

- `registerUser`
- `loginUser`
- `logoutUser`
- `refreshAccesstoken`
- `changecurrentUserpassword`
- `getCurrentUser`
- `updateAccountdetails`
- `updateUseravatar`
- `updateUsercoverimage`
- `getUserchannelprofile`
- `getwatchhistory`

Route: `routes/user.routes.js`

Base path: `/api/v1/users`

1. POST `/register`
 - Multer fields:
 - `avatar` (max 1)
 - `coverimage` (max 1)
 - Controller: `registerUser`
2. POST `/login`
 - Controller: `loginUser`
3. POST `/logout`
 - Middleware: `verifyJWT`
 - Controller: `logoutUser`
4. POST `/refresh-token`

- Controller: `refreshAccessToken`

5. `POST /change-password`

- Middleware: `verifyJWT`
- Controller: `changecurrentUserpassword`

6. `GET /current-user`

- Middleware: `verifyJWT`
- Controller: `getCurrentUser`

7. `PATCH /update-account`

- Middleware: `verifyJWT`
- Controller: `updateAccountdetails`

8. `PATCH /avatar`

- Middleware: `verifyJWT`, Multer single: `avatar`
- Controller: `updateUseravatar`

9. `PATCH /cover-image`

- Middleware: `verifyJWT`, Multer single: `coverimage / coverImage` mismatch in code
- Controller: `updateUsercoverimage`

10. `GET /c/:username`

- Middleware: `verifyJWT`
- Controller: `getUserchannelprofile`

11. `GET /history`

- Middleware: `verifyJWT`
- Controller: `getwatchhistory`

Key user logic

1) `registerUser`

Flow:

1. Read body: `fullname, email, username, password`
2. Validate non-empty

3. Check existence:

- `User.findOne({ $or: [{email}, {username}] })`

4. Extract uploaded files from `req.files` :

- `avatar` is required
- `coverimage` optional

5. Upload to Cloudinary:

- `uploadoncloudinary(avatarLocalPath)`
- optionally upload cover image

6. Create user:

- stores avatar url, coverimage url or empty string
- normalizes username to lowercase

7. Response:

- `201` with `ApiResponse` containing `createdUser`

2) `loginUser`

Flow:

1. Accept `username` or `email` (at least one required)
2. Find user via `$or`
3. Validate password using `user.isPasswordcorrect(password)`
4. Generate tokens using helper `generateAccessandRefreshToken(user._id)` :
 - creates access token and refresh token using user model methods
 - stores new refresh token in `user.refreshToken`
5. Fetch logged-in user excluding password/refreshToken
6. Set cookies:
 - `accesstoken` and `refreshToken`
 - `httpOnly: true, secure: true`
7. Response:
 - `200` with `ApiResponse` data: `{ user, accesstoken, refreshToken }`

3) `logoutUser`

Flow:

- Updates current user's `refreshToken` to `undefined`
- Clears cookies `accesstoken` and `refreshToken`

4) `refreshAccesstoken`

Flow:

1. Reads refresh token from cookies `req.cookies.refreshtoken` or body
2. Verifies JWT using `REFRESH_TOKEN_SECRET`
3. Loads user by decoded `_id`
4. Checks token rotation:
 - if frontend token differs from `user.refreshtoken` → error
5. Generates new access + refresh token
6. Sets cookies again

5) `getUserchannelprofile` (aggregation)

- Parameter: `:username`
- Uses MongoDB aggregation on `User` :
 1. `$match` by `username`
 2. `$lookup` subscribers from `subscriptions` where local `_id` matches `channel`
 3. `$lookup` subscribedto from `subscriptions` where local `_id` matches `subscriber`
 4. `$addFields` :
 - `subscribersCount` = size of subscribers array
 - `channelsubscribedtocount` = size of subscribedto array
 - `issubscribed` = whether `req.user._id` appears in `subscribers.subscriber`
 5. `$project` returns specific fields
- Returns first channel document

6) `getwatchhistory` (aggregation)

- Aggregates starting from the current user:
 - `$match` user `_id`
 - `$lookup` into `videos` using `watchHistory` array
 - inner pipeline:
 - `$lookup` owner details from `users`
 - `$project` owner fields
- Returns `user[0].watchHistory`

7) Password and account update helpers

- `changecurrentUserpassword` :
 - verifies old password via `user.isPasswordcorrect`
 - sets `user.password = newpassword` and saves
- `updateAccountdetails` :
 - `findByIdAndUpdate` of `fullname/email`

- returns without password
- `updateUseravatar` and `updateUsercoverimage` :
 - upload local file to Cloudinary and update user

Known mismatches / issues in user code

- `user.controller.js` uses `uploadoncloudinary` but `video.controller.js` imports `uploadOnCloudinary` (capitalization mismatch).
- In `routes/user.routes.js` `cover-image` uses `upload.single("coverImage")` but controller expects `req.file?.path` (name mismatch may break upload unless middleware uses that field name exactly).
- `verifyJWT` expects cookie name `accessToken` , but login sets cookie `accesstoken` (spelling mismatch). This will affect auth.

These are important for "A-to-Z" correctness when implementing/using endpoints.

9.3 Video controller & routes (mostly TODO skeleton)

Controller: `controllers/video.controller.js`

Exports:

- `getAllVideos`
- `publishAVideo`
- `getVideoById`
- `updateVideo`
- `deleteVideo`
- `togglePublishStatus`

Current state:

- Most methods contain only TODO comments.

Route: `routes/video.routes.js`

- Base path: `/api/v1/videos`
- This backend's current `video.routes.js` actually includes **user controller** routes (register/login/etc) and is misnamed.

Routes in `routes/video.routes.js` (as discovered):

- `/register`, `/login`, `/logout`, `/refresh-token`, `/change-password`, `/current-user`, `/update-account`, `/avatar`, `/cover-image`, `/c/:username`, `/history`

So the effective endpoints for user auth/profile are mounted under `/api/v1/videos/*` due to `app.js` mounting `videoRouter` at `/api/v1/videos`.

This is a structural bug in the repo. In `app.js`, `userRouter` is mounted at `/api/v1/users`, but the file we loaded named `video.routes.js` contains user routes; actual `user.routes.js` also exists and contains user routes.

9.4 Comment controller & routes (TODO skeleton)

Controller: `controllers/comment.controller.js`

Exports:

- `getVideoComments`
- `addComment`
- `updateComment`
- `deleteComment`

Most are TODO.

Route: `routes/comment.routes.js`

Base mount: `/api/v1/comments`

- `router.use(verifyJWT)` applied to all
- `GET /:videoId` → `getVideoComments`
- `POST /:videoId` → `addComment`
- `DELETE /c/:commentId` → `deleteComment`
- `PATCH /c/:commentId` → `updateComment`

9.5 Like controller & routes (TODO skeleton)

Controller: `controllers/like.controller.js`

Exports:

- `toggleVideoLike`
- `toggleCommentLike`

- `toggleTweetLike`
- `getLikedVideos`

All TODO.

Route: `routes/like.routes.js`

Base mount: `/api/v1/likes`

- Auth for all routes via `router.use (verifyJWT)`
- Endpoints:
 - `POST /toggle/v/:videoId`
 - `POST /toggle/c/:commentId`
 - `POST /toggle/t/:tweetId`
 - `GET /videos`

9.6 Playlist controller & routes (TODO skeleton)

Controller: `controllers/playlist.controller.js`

Exports:

- `createPlaylist`
- `getUserPlaylists`
- `getPlaylistById`
- `addVideoToPlaylist`
- `removeVideoFromPlaylist`
- `deletePlaylist`
- `updatePlaylist`

All TODO.

Route: `routes/playlist.routes.js`

Base mount: `/api/v1/playlist`

- Auth for all routes via `router.use (verifyJWT)`
- Endpoints:
 - `POST /` → create playlist
 - `GET /:playlistId`
 - `PATCH /:playlistId`

- `DELETE /:playlistId`
- `PATCH /add/:videoId/:playlistId`
- `PATCH /remove/:videoId/:playlistId`
- `GET /user/:userId`

9.7 Subscription controller & routes (TODO skeleton)

Controller: `controllers/subscription.controller.js`

Exports:

- `toggleSubscription`
- `getUserChannelSubscribers`
- `getSubscribedChannels`

All TODO.

Route: `routes/subscription.routes.js`

Base mount: `/api/v1/subscriptions`

- Auth for all via `router.use(verifyJWT)`
- Endpoints:
 - `GET /c/:channelId` → `getSubscribedChannels` (naming suggests mismatch but uses that function)
 - `POST /c/:channelId` → `toggleSubscription`
 - `GET /u/:subscriberId` → `getUserChannelSubscribers`

9.8 Dashboard controller & routes (TODO skeleton)

Controller: `controllers/dashboard.controller.js`

Exports:

- `getChannelStats`
- `getChannelVideos`

TODO.

Route: `routes/dashboard.routes.js`

Base mount: `/api/v1/dashboard`

- Auth for all via `router.use (verifyJWT)`
- Endpoints:
 - `GET /stats`
 - `GET /videos`

9.9 Tweet controller & routes (TODO skeleton)

Controller: `controllers/tweet.controller.js`

Exports:

- `createTweet`
- `getUserTweets`
- `updateTweet`
- `deleteTweet`

TODO.

Route: `routes/tweet.routes.js`

Base mount: `/api/v1/tweets`

- Auth for all routes via `router.use (verifyJWT)`
 - Endpoints:
 - `POST /`
 - `GET /user/:userId`
 - `PATCH /:tweetId`
 - `DELETE /:tweetId`
-

10. Step-by-step repository

(Youtube_Clone_Backend_STEPS/)

This folder is an **educational sequence** explaining how the final backend evolves.
It contains:

- `Backend.txt` / `HTTP.txt` / `README.md`
- Multiple step folders with their own `flow.txt` and incremental code

10.1 `Backend.txt` (core mental model)

- Backend = **Language/framework** + **Database/ORM/ODM**
 - Browser/Mobile ⇌ API ⇌ Backend ⇌ Database (remote latency)
 - For Node-based backend: deals with **Data + File + Third Party APIs**
 - Suggested structure: `src/index` (DB connect), `src/app` (express config), `constants` (DB-name), etc.
-

10.2 Step flows (`flow.txt`) that document “how the backend works”

Below are the key ideas extracted from the `flow.txt` files we read.

1_Connect_Frontend_with_Backend/flow.txt

- Explain basics of HTTP + headers.
- Explain frontend/backend are different origins (protocol + domain + port).
- CORS is a **browser security rule**.
- During development, use **Vite proxy** to avoid CORS: proxy `/api` → backend `http://localhost:3000`.
- Production still needs correct backend CORS config.
- Explains HTTP methods & status codes at a conceptual level.

2_Data_Modeling_For_Backend_With_Mongoose/flow.txt

- Emphasizes **data modeling**: define what fields you store first (register model), then build login/validation on top.
- Mentions using modeling tools (Moon Modeler, eraser.io).

3_Professional_Project_Setup/flow.txt

- Setup for production-grade project hygiene:
 - `.gitkeep` for empty dirs
 - `.gitignore`
 - `.env` usage (don't commit secrets)
 - `"type": "module"`
 - nodemon for dev
 - prettier config files (`.prettierrc` , `.prettierignore`)

4_Connect_Database_in_MERN_Debugging/flow.txt

- Setup `.env` with port + MongoDB URI
- Connect DB with mongoose using async/promise patterns (try/catch)

5_Custom_APIResponse_ErrorHandling/flow.txt

- Introduces cookie-based auth concepts:
 - `cookie-parser` to read cookies from the browser
 - `cors` to allow cross-origin calls
- Explains Express middleware signature: `(err, req, res, next)` and `next()`
- Mentions building structured API error handling

6_User_Video_with_Hooks_JWT/flow.txt

- JWT concepts:
 - Access token short-lived
 - Refresh token long-lived
 - Store refresh token in DB (session-like)
- Token lifecycle:
 - Validate access token for protected features
 - If access token expires (401), call refresh endpoint with refresh token

7.Upload_File_in_Backend/flow.txt

- Cloudinary = third-party SDK for file storage
- Upload pipeline:
 1. Client uploads file
 2. Multer temporarily stores locally on server
 3. Cloudinary uploads from local temp folder
- Mentions installing `cloudinary` + `multer`
- Notes: create helpers in `utils` + inject multer middleware where needed

8_Routers_and_Controllers/flow.txt

- Uses Postman to test register flow
- Explains multipart form-data limitations:
 - files must be sent as form-data keys
- After sending request:
 - verify in temp upload folder
 - verify in Cloudinary media explorer
 - delete failed uploads and debug

9_Register_Controller/flow.txt

- (Short snippet read during this session) Postman testing guidance for registration + file upload.

10_AccessRefreshToken_Middleware_Cookies/flow.txt

- Explains refresh token rotation logic at a conceptual level
- How the session continues without re-login:
 - access token invalid → user calls refresh endpoint
 - backend verifies refresh token matches DB stored refresh token
 - backend issues new tokens

11_Mongodb_Aggregation_Pipelines/flow.txt

- The file content is currently empty in this repo snapshot (read returned blank).

How to use this documentation effectively

- Treat **Youtube_Clone_Backend/** as the **final target**.
- Treat **Youtube_Clone_Backend_STEPS/** as the **learning path** showing how:
 - Express bootstrapping
 - Mongoose modeling
 - JWT + cookie auth
 - Cloudinary file uploads
 - Aggregationsevolve across iterations.

11. Environment variables (.env)

The final backend expects variables referenced throughout the code:

Auth

- `ACCESS_TOKEN_SECRET`
- `REFRESH_TOKEN_SECRET`
- `ACCESS_TOKEN_EXPIRY`
- `REFRESH_TOKEN_EXPIRY`

Database

- `MONGODB_URI`

CORS

- `CORS_ORIGIN` (optional)

Cloudinary

- `CLOUDINARY_CLOUD_NAME`
 - `CLOUDINARY_API_KEY`
 - `CLOUDINARY_API_SECRET`
-

12. Endpoint summary (effective mounts)

Because `app.js` mounts routers under `/api/v1/*`, the intended endpoints are:

- `/api/v1/healthcheck/` (GET)
 - `/api/v1/users/*` (auth/profile/video-like user features)
 - `/api/v1/tweets/*` (auth protected)
 - `/api/v1/subscriptions/*` (auth protected)
 - `/api/v1/videos/*` (video CRUD skeleton, but also check routing file misnaming issues)
 - `/api/v1/comments/*` (auth protected)
 - `/api/v1/likes/*` (auth protected)
 - `/api/v1/playlist/*` (auth protected)
 - `/api/v1/dashboard/*` (auth protected)
-

13. Security considerations and correctness issues

13.1 Cookie and token name mismatches

There is likely a mismatch between how access token cookie is set and how it is read.

- `loginUser()` sets cookie named `accesstoken`
- `verifyJWT` reads cookie named `accessToken`

This can prevent auth from working.

13.2 `routes/user.routes.js` cover image field name mismatch

- `upload.single("coverImage")` vs `registerUser` expecting `req.files.coverimage`

This can break cover uploads depending on how frontend sends multipart form fields.

13.3 `video.routes.js` file naming mismatch

`app.js` mounts `videoRouter` at `/api/v1/videos`, but `routes/video.routes.js` content we inspected appears to contain **user routes**.

This creates routing confusion.

14. MongoDB aggregation pipelines used

14.1 `getUserchannelprofile`

- Shows how to compute:
 - subscriber count
 - subscribed-to count
 - whether requesting user is subscribed to the channel

14.2 `getwatchhistory`

- Shows how to:
 - join videos by `watchHistory` array
 - also join each video owner details
-

15. What's complete vs TODO (final backend)

Implemented (meaningful logic)

- User auth flows:
 - register, login, logout
 - access refresh rotation
 - change password
 - update account details
 - avatar + cover upload
 - channel profile aggregation
 - watch history aggregation

Skeleton / TODO

- video CRUD and publishing
 - comment CRUD
 - like toggling
 - playlist management
 - subscription toggling
 - dashboard stats and videos
 - tweet CRUD
 - healthcheck response
-

16. Suggested improvements (based on code inspection)

1. Fix cookie naming consistency (`accesstoken` vs `accessToken`).
 2. Fix cover upload field name consistency (`coverImage` vs `coverimage`).
 3. Verify router file naming and mounting correctness:
 - `routes/video.routes.js` should contain video endpoints; user endpoints should remain in `routes/user.routes.js` .
 4. Complete TODO controller methods.
-

17. Deliverable created

This file is generated at:

- `Youtube_Clone_Backend_DOCS.md`
-

End of documentation.